

SysMD Notebook Kickstart

Technische Universität Kaiserslautern
Fachbereich Informatik
Lehrstuhl Cyber-Physical Systems

[1 Download, Installation und Start](#)

[2 SysMD Notebook starten und erste Aktionen](#)

[3 Erstes Modell erstellen](#)

[3.1 Erstellen einer SysMD Datei](#)

[3.2 Hinzufügen und Löschen von Modell-Elementen](#)

[Hinzufügen, Komprimierte Darstellung und Löschen](#)

[Editieren von Elementen](#)

[3.3 Analyse und Ausgabe der Ergebnisse \(oder Fehler\)](#)

[4 Die Sprache SysMD](#)

[4.1 Modellierung von Klassen bzw. Varianten](#)

[4.2 Eigenschaften und Merkmale von Klassen](#)

[4.3 Typen und Funktionen in Expressions](#)

[Funktionen über Aggregation von Teilen / SUM-Funktion](#)

[Funktionen über Subklassen von Teilen / bySubclasses-Funktion](#)

[4.4 Vererbung](#)

[4.5 Intervall-Semantik in Spezifikationen](#)

[5 “Hybrid Spaces” und SysML v2 Textual](#)

[5.1 Unterstützte Sprachen: Markdown, SysMD, SysML v2 Textual](#)

1 Download, Installation und Start

Eine aktuelle Version von SysMD Notebook kann von der Edacentrum "Owncloud" heruntergeladen. Sie sind im Ordner

Verwertung/Tutorials/Agila

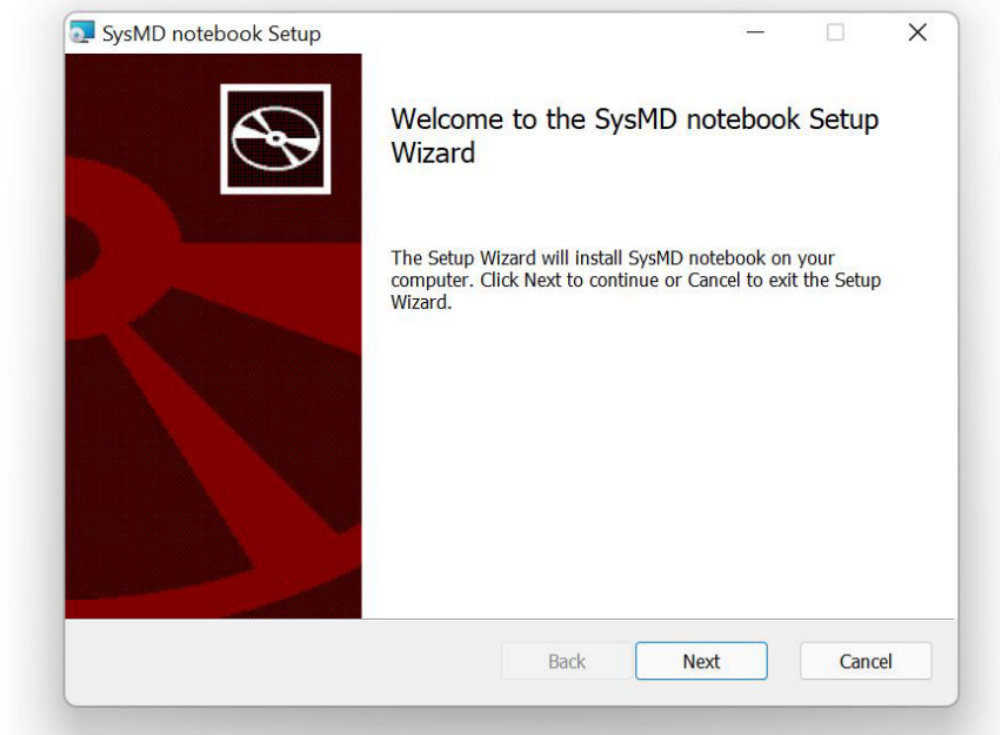
Oder unter folgender URL zu finden:

<https://owncloud.edacentrum.de/index.php/apps/files/?dir=/genial!/Verwertung/Tutorials/>

Zur Installation wird eine Installationsdatei abhängig vom verwendeten Betriebssystem benötigt:

Betriebssystem	Datei
MS Windows	SysMDnotebook[versionsnummer].msi
Mac OS X; M1 Prozessor	SysMDnotebook[versionsnummer].dmg
Intel Prozessor	(auf Nachfrage)
Linux	(auf Nachfrage)

Nach dem Download die Datei anklicken. Es startet ein "Installer" mit Setup-Wizard.



Bei Windows kann es je nach Version und Voreinstellungen zu einer Nachfrage kommen:

Der Computer wurde durch Windows geschützt

Von Microsoft Defender SmartScreen wurde der Start einer unbekannten App verhindert. Die Ausführung dieser App stellt u. U. ein Risiko für den PC dar.
[Weitere Informationen](#)

Hier können wir *“Weitere Informationen”* wählen. Anschließend immer *“Next”* oder *“OK”* wählen, bis SysMD Notebook installiert ist - fertig!

2 SysMD Notebook starten und erste Aktionen

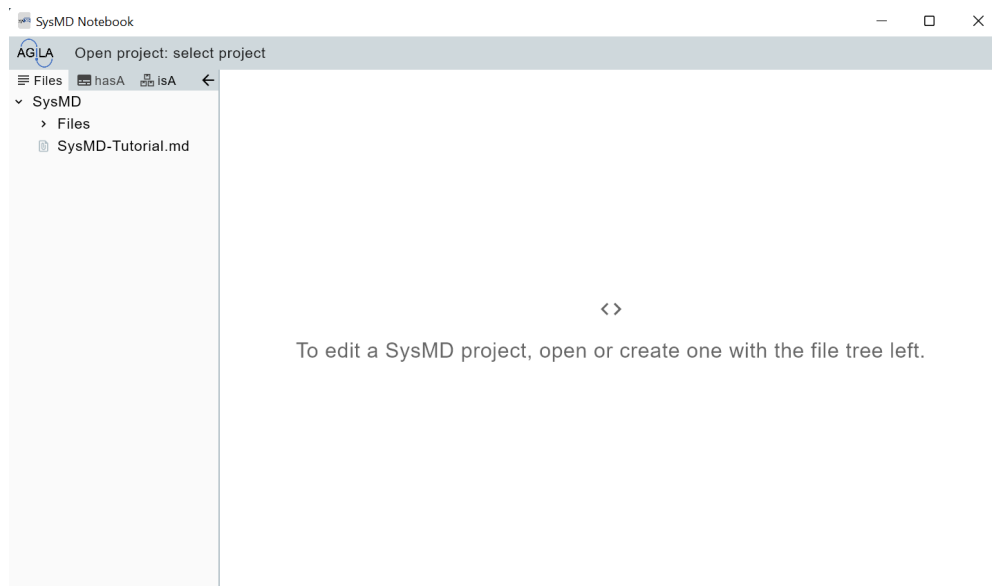
Der Installer von SysMD Notebook hat verschiedene Möglichkeiten zum Start installiert (je nach Wahl):

- Start durch ein Icon auf dem Desktop
- Start bei allen Apps - zu finden dann unter *“SysMD Notebook”*



*Nach dem Start installiert SysMD ein Verzeichnis **“Eigene Dateien/SysMD”**. Hier werden Ihre editierten Dateien gespeichert oder gefunden.*

Ein leeres SysMD Notebook sieht nach dem Start wie folgt aus:



Der linke Bereich ist eine *“Treeview”*, der zur Navigation dient. Hier kann man

- 1) Dateien auswählen und laden.
- 2) Inhalte der Datenbank und Analyseergebnisse betrachten. Mehr dazu später.

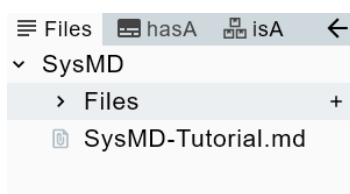
Im rechten Bereich können wir unsere Modelle und Dokumente editieren.

3 Erstes Modell erstellen

Zum Erstellen eines Modells gehen wir wie folgt vor:

- 1) Erstellen einer SysMD-Datei.
- 2) Hinzufügen von Modell-Elementen:
 - Dokumentation in Markdown
 - Modell in "SysMD" oder "SysML v2 textual".

3.1 Erstellen einer SysMD Datei



Um ein erstes Modell zu erstellen, müssen wir zunächst eine Datei erstellen. Hierzu verwenden wir die Treeview "Files" und navigieren zu dem kleinen "+". Wenn wir es anwählen, erstellen wir eine neue Datei.

SysMD Notebook fragt nun nach einem Dateinamen - wir können hier einen Namen mit der Endung ".md" eingeben und mit "Return" bestätigen.

Die neue Datei befindet sich dann im Ordner "SysMD/Files" in unserem Verzeichnis "Eigene Dateien" und ist erst einmal leer.

3.2 Hinzufügen und Löschen von Modell-Elementen

Ein SysMD/SysMLv2 - Modell ist in *Elemente* strukturiert. Ein Element kann dabei sein:

- *Dokumentation*. In SysMD Notebook im Markdown-Format. Dies kann Tabellen und Bilder enthalten. Somit können wir ein Modell besser und einfacher erklären - und andere können das Modell schneller verstehen.
- *Textuelles Modell*. Textuelle Modelle können in der Modellierungssprache SysMD oder einem Subset der SysML v2 Textual beschrieben werden. Beides verwendet letztlich die KerML Klassen von SysMV v2 und die SysML v2 API.

Hinzufügen, Komprimierte Darstellung und Löschen

Zum Hinzufügen von Elementen klicken wir einfach auf den Platz zwischen zwei Elementen oder vor/nach dem ersten/letzten Element. Dieser ist mit einem kleinen grauen Kreis kenntlich gemacht. Zu Beginn gibt es natürlich noch keine Elemente. Wir wählen daher einfach den einzigen Kreis aus:



Nun ist auch schon unser erstes Element entstanden. Links vom Modell werden nun Icons angezeigt. Diese stellen - abhängig vom Typ des Elements - mögliche Aktionen dar. Zum Löschen eines Elements wählt man den Mülleimer. Zum

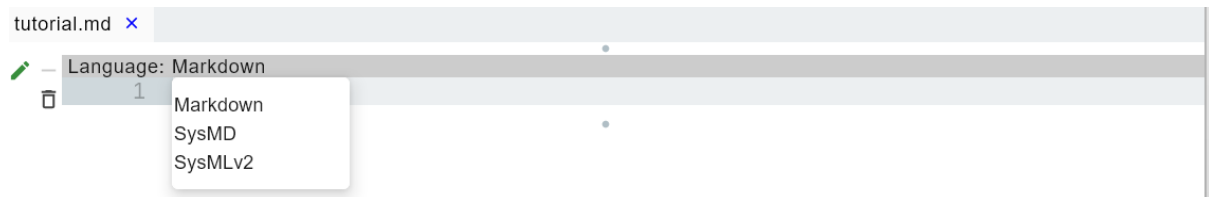


Editieren den Stift. Um ein großes Element komprimiert darzustellen wählt man das “-”.

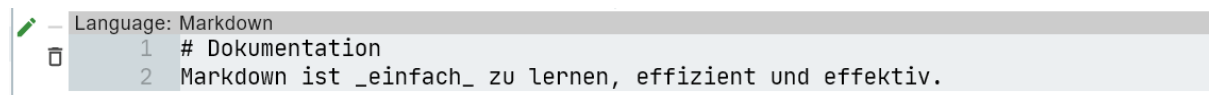
Editieren von Elementen

Um ein Element zu editieren klicken wir auf den kleinen Stift links oder mit einem Doppelklick in diese Modell. Nun haben wir die Möglichkeit, dieses Element zu editieren.

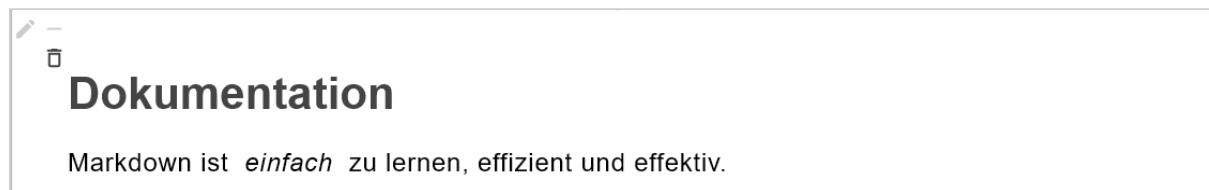
Hierzu muss zunächst die Art des Modellelements gewählt werden. Diese können wir wählen, wenn wir ein Element aktivieren (Doppelklick auf das Element oder einfacher Klick auf den Stift). Oben erscheint dann ein Menü, in dem wir die Sprache wählen können:



Um eine **Dokumentation** zu erstellen, wählen wir “Markdown” und geben ein:

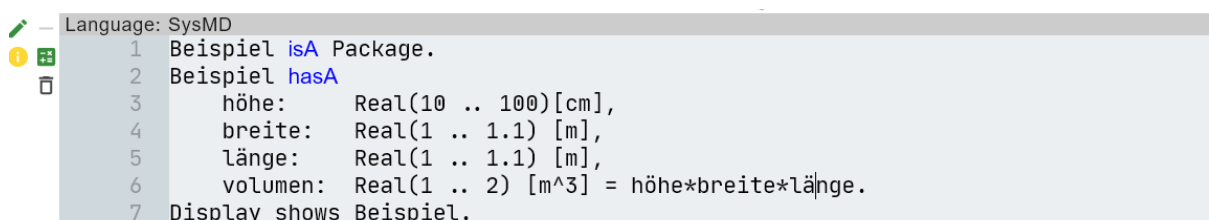
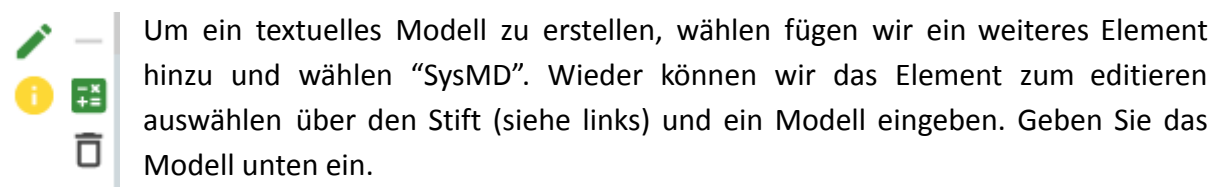


Nun wählen wir nochmal den Stift aus, um den Editiermodus zu verlassen. Nun rendert SysMD das Dokument:



Markdown ermöglicht neben der Strukturierung in Überschriften, Unterüberschriften auch das Erstellen von Tabellen, das Einfügen von Bildern und vieles mehr. Für eine umfassende Einführung in Markdown sei auf folgende URL verwiesen:

<https://commonmark.org/help/tutorial/>



3.3 Analyse und Ausgabe der Ergebnisse (oder Fehler)



Um die Ergebnisse eines Elements und auch Fehler – ggf. auch in anderen Elementen – angezeigt zu bekommen wählen wir das “i” für “Information” aus.

Danach (oder davor - die Reihenfolge spielt da keine Rolle) klicken wir auf den Taschenrechner. SysMD Notebook zeigt uns nun die möglichen Werte der einzelnen Variablen:

```
Language: SysMD
1 Beispiel isA Package.
2 Beispiel hasA
3   höhe:      Real(10 .. 100) [cm],
4   breite:    Real(1 .. 1.1) [m],
5   länge:     Real(1 .. 1.1) [m],
6   volumen:   Real(1 .. 2) [m^3] = höhe*breite*länge.
7 Display shows Beispiel.
```

In Beispiel:
höhe = 82.64463..100 cm
breite = 1..1.1 m
länge = 1..1.1 m
volumen = 1..1.21 m^3

Wer lieber mit standardisierten Sprachen arbeitet, kann auch “SysML” auswählen. Dann kann alternativ die Sprache SysML v2 textual verwendet werden.

4 Die Sprache SysMD

SysMD wurde entwickelt, um für Ingenieure verschiedenster Disziplinen unmittelbar und ohne Einarbeitung anwendbar zu sein:

Lesbarkeit: Anweisungen sind normalen Sätzen nachempfunden. Aufzählungen werden durch Kommata getrennt; Anweisungen enden mit einem Punkt.

Erlernbarkeit: Die Anzahl der Regeln wurde bewusst klein gehalten und die Regeln finden sich immer in gleicher form wieder.

Klarheit: Die Spezifikation von Werten oder Bereichen wurde explizit von der Formulierung mathematischer Abhängigkeiten getrennt.

Interoperabilität: Zur internen Darstellung wird das KerML der SysML v2 textual verwendet; zur Austausch kann über das SysML v2 API darauf zugegriffen werden. Da die Standardisierung von SysML v2 noch nicht abgeschlossen ist und wir auch nur eine kleine Teilmenge unterstützen, fokussieren wir uns zunächst auf SysMD.

4.1 Modellierung von Klassen bzw. Varianten

Zur Modellierung von Varianten erstellen wir Taxonomien. Das sind Bäume wie in der Informatik: aus einer Wurzel oben, Knoten und Blättern ganz unten. In Taxonomien sind Wurzel, Knoten und Blätter Klassen die wir ordnen:

- Die Wurzel stellt die allgemeinste Klasse dar.
- Zu den Blättern hin werden Klassen sukzessive spezieller.

In SysMD gibt es eine ausgezeichnete Klasse, die Wurzel aller Taxonomie - Klassen ist: "Any". Zur Modellierung einer Taxonomie verwenden wir die Relation "is a" (deutsch: ist ein") bzw. in SysMD mit dem Schlüsselwort "isA".

Eine Taxonomie von Fahrzeugen könnte wie folgt aussehen:

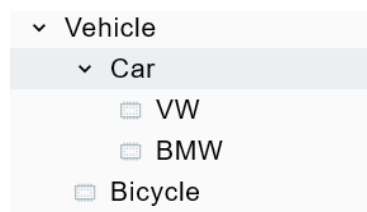
```
Language: SysMD
1 Vehicle isA Any.
2 Car isA Vehicle.
3 Bicycle isA Vehicle.
4 VW isA Car.
5 BMW isA Car.
```

Das ist möglich, wird aber unübersichtlich wenn wir einige hundert Klassen haben. Deshalb Sinnvoll ist es deshalb, Definitionen *hierarchisch* zu strukturieren in einer "Package", die Definitionen enthält.

Die zusammengehörigen Definitionen sind durch Semikolon getrennt und die Definitionen enden mit einem Punkt:

```
Language: SysMD
1 Autos isA Package.
2 Autos defines
3   Vehicle isA Any;
4   Car isA Vehicle;
5   Bicycle isA Vehicle;
6   VW isA Car;
7   BMW isA Car. |
```

Die entsprechenden Definitionen tauchen nun in der "Tree-View" unter der Relation "isA" auf:



Wir verwenden Taxonomien um die Wissensbasis zu strukturieren. Ziel ist, allgemeine Klassen einzusetzen und später zu prüfen, welche Spezialisierungen für in einem bestimmten Kontext geeignet sind. Hierbei spielt Vererbung eine Rolle.

4.2 Eigenschaften und Merkmale von Klassen

Klassen haben bestimmte Merkmale (engl. Features). Diese modellieren wir in SysMD mit der "hasA" Relation. Wir definieren uns hierzu zunächst einen Katalog von Autoteilen. Auf die Elemente dieser Package können wir zugreifen, indem wir den Namen der Package und den des Bauteils, getrennt durch zwei Doppelpunkte verwenden - wie im Beispiel.

```
Language: SysMD
1 Autoteile isA Package.
2 Autoteile defines
3   Engine isA Component;
4   Wheels isA Component.
5
6 Autos::Car hasA
7   wheels: [4 .. 4] Autoteile::Wheels,
8   engine: [1 .. 2] Autoteile::Engine,
9   mass: Real (100 .. 1000) [kg].
10
```

Anschließend können wir Aussagen über zum Beispiel ein Auto machen. Hierzu werden die "Features" mit Hilfe der *hasA*-Relation aufgezählt, und zwar in folgender Reihenfolge:

- Name (z.B. "wheels"), gefolgt von einem Doppelpunkt
- Multiplizität, also ein Intervall das die mögliche Anzahl spezifiziert ([4..4])
- Spezifizierte Klasse ("Autoteile::Wheels")

Neben Features können auch Eigenschaften modelliert werden, wie zum Beispiel die Masse. Eigenschaften werden als spezielles Feature modelliert. Es wird angegeben:

- Name (z.B. mass)
- Spezifizierte Klasse und Einschränkungen (z.B. Real (100 .. 100))
- Einheit in eckigen Klammern (z.B. [kg])

Einige SysMD-Notebooks können Anregungen geben:

```
Language: SysMD
1 SiEinheitenBeispiel isA Package.
2
3 SiEinheitenBeispiel hasA
4   t: Real [s] = 1.0 [s],
5   v: Real [m/s] = 3.0 [m/s],
6   g: Real [m/s^2] = 4.0 [m/s^2],
7   s: Real [m/s] = sqrt(sqr(v)+sqr(g)*sqr(t)).
8
9 Display shows SiEinheitenBeispiel.
```

In SiEinheitenBeispiel:

```
t = 1..1 s
v = 3..3 m / s
g = 4..4 m / s^2
s = 5..5 m / s
```

SysMD unterstützt die SI Einheiten. Einheiten werden in eckigen Klammern notiert und in Rechnungen automatisch konvertiert und auf Konsistenz geprüft.

Die Einheit links einer Abhängigkeit (=) muss in die rechts überführbar sein.

```
Language: SysMD
1 DateTime isA Package.
2
3 DateTime hasA
4   date: Real[DateTime] = DateTime("2021-10-10T03:00:00"),
5   time: Real[a] = 1.0 a,
6   dateResult: Real[DateTime] = date + time.
7
8 Display shows DateTime.
```

In DateTime:

```
date = 2021-10-10T03:00
time = 1..1 a
dateResult = 2022-10-10T03:00
```

Datum und Zeit werden im ISO-Format notiert.

Man kann Datum und Zeit addieren oder subtrahieren.

4.3 Typen und Funktionen in Expressions

Die folgenden Datentypen werden unterstützt:

- Real (Intervalle)
- Integer (Intervalle) – Nicht verwenden für IRIS-Modelle.
- Boolean
- String – Nicht verwenden für IRIS-Modelle.

Es werden in den Expressions die folgenden Funktionen unterstützt (v2.5.20+)

- *ceil* - aufrunden von Real x auf Integer– Nicht verwenden für IRIS-Modelle.
- *floor(x)* - abrunden von Real x auf Integer– Nicht verwenden für IRIS-Modelle.
- *exp(x)* - Exponentialfunktion von x
- *log(x)* - Logarithmus von x
- *power2* – Nicht verwenden für IRIS-Modelle.
- *powerb* – Nicht verwenden für IRIS-Modelle.
- *sqr(x)* - Quadrat von x
- *sqrt(x)* - Quadratwurzel von x
- *linear(a, b, c, d, ...)* – lineare Interpolation, wobei paarweise Punkte spezifiziert werden, zwischen denen interpoliert wird.
- *ITE(condition, if, else)* – ITE Funktion; wenn Bedingung wahr ist, wird if berechnet, andernfalls else.
- *sum_i(...)* Iteration – Nicht verwenden für IRIS-Modelle.
- *not(x)*
- *a and b*
- *a or b*

Funktionen über Aggregation von Teilen / SUM-Funktion

Das Beispiel der Autoteile-Bibliothek lässt sich erweitern. Und zwar können wir jedem Teil eine Masse - ggf. auch ein Intervall - zuordnen. Oft ergeben sich Eigenschaften eines

Systems aus den Eigenschaften seiner Teile.

Im Beispiel links wird durch die Funktion "SUM(mass)" die Summe aller Teile (und ggf. deren Teile) berechnet.

```
Language: SysMD
1 Autoteile isA Package.
2 Autoteile defines
3   Body isA Component;
4   Engine isA Component;
5   Wheel isA Component.
6
7 Autoteile::Engine hasA mass: Real(200.0) [kg].
8 Autoteile::Wheel hasA mass: Real(50.0) [kg].
9 Autoteile::Body hasA mass: Real(100.0) [kg].
10
11 Autos::Car hasA
12   body: Autoteile::Body,
13   wheels: [4 .. 4] Autoteile::Wheel,
14   engine: [1 .. 2] Autoteile::Engine,
15   mass: Real [kg] = SUM(mass).
16
17 Display shows Autos::Car::mass.
```

mass = 500..700 kg

Funktionen über Subklassen von Teilen / bySubclasses-Funktion

Wenn wir die Eigenschaften einer Superklasse nicht selber definieren wollen, sondern sie sich aus den Eigenschaften aller bekannten Subklassen ergeben soll, können wir das über die Funktion `bySubclasses(name)` tun.

In der Autos-Bibliothek können wir damit zum Beispiel die Masse von Fahrzeugen allgemein festlegen:

```
Language: SysMD
1
2 Autos::Bicycle hasA mass: Real(10 .. 20) [kg].
3 Autos::Vehicle hasA mass: Real = bySubclasses(mass).
4 Display shows Autos::Vehicle::mass.

mass = 10..700
```

4.4 Vererbung

In Taxonomien vererbt SysMD Eigenschaften ihrer übergeordneten, allgemeinen Klasse (Superklasse) an ihre Spezialisierungen. Hierbei orientiert sich die definierte Semantik am Liskov-Prinzip:



“Eine Spezialisierung soll immer ihre Superklasse ersetzen können”.

In unserer Bibliothek heisst das, dass wir wenn wir an einer Stelle ein “Vehicle” wollen, wir zunächst mit einem Fahrzeug beginnen können. Gültige Alternativen sind dann auch alle Subklassen von “Vehicle”

- Bicycle
- Car, sowie dessen Spezialisierungen
 - VW
 - BMW

Damit dies immer möglich ist, werden in SysMD alle Eigenschaften “identisch” vererbt. Natürlich können wir die Eigenschaften ändern, d.h. ein VW und ein BMW können sich von einem allgemeinen “Car” unterscheiden. Aber immer nur so, dass das Liskov-Prinzip gilt. Entsprechend erben Spezialisierungen alle Eigenschaften ihrer Superklassen.

```
Language: SysMD
1 Autos::Car hasA power: Real(10 .. 1000) [kW].
2 Autos::VW hasA power: Real(20 .. 100) [kW].
3 Autos::BMW hasA power: Real(150 .. 400) [kW].
```

Ändern wir zum Beispiel die *power* von VW zu max. 1100 W, so widerspricht das Aussage in Zeile 1, dass Fahrzeuge (Car) eine Leistung im Bereich von 10 bis 1000 kW haben. Hier lässt sich z.B. die Funktion *bySubclasses* vorteilhaft anwenden.

```
Language: SysMD
1 Autos::Car hasA power: Real(10 .. 1000) [kW].
2 Autos::VW hasA power: Real(20 .. 1100) [kW].
3 Autos::BMW hasA power: Real(150 .. 400) [kW].
```

ERROR: In power: column 29: INCONSISTENCY: subclass value 20..1100 of power must be refinement of superclass value 10..1000

4.5 Intervall-Semantik in Spezifikationen

Intervalle in Spezifikationen könne unterschiedliche Semantiken haben. Zum Beispiel können wir spezifizieren, dass für ein Teil ein bestimmter (beliebiger) Wert aus einem Intervall gewählt werden soll. Dies wäre zum Beispiel bei der “Power” im Beispiel oben der Fall:

Wenn wir sagen

power: Real(20 .. 100) [kW]

meinen wir damit, wir haben bzw. wollen ein Fahrzeug, in dem power einen Wert aus dem Intervall von 20 bis 100 kW besitzt. 10 kW wäre also außerhalb der Spezifikation.

Diese Semantik wäre aber falsch, wenn wir einen erforderlichen Temperatureinsatzbereich von -20 bis 60 °C modellieren. Dann wäre

temperature: Real(-20 .. 60) [°C]

schlicht falsch. Denn damit wäre ein Fahrzeug mit einem Temperaturbereich von 0 bis 40 °C eine gültige Verfeinerung. In SysMD gibt es darum noch das Keyword “all”. Es spezifiziert, dass eine Abhängigkeit für alle Werte immer erfüllt sein muss. Also in SysMD:

temperature: all Real(-20 .. 60) [°C]

Dies hat Auswirkungen auf

- Constraint-Propagation
- Vererbung

Hierbei muss die andere Semantik berücksichtigt werden.

5 “Hybrid Spaces” und SysML v2 Textual

Wer am Austausch von Modellen interessiert ist, kommt an Standards nicht vorbei. SysMD Notebook unterstützt das Konzept von “Hybrid Spaces”. Damit ist gemeint, dass Requirements und Spezifikationen - je nach Anforderung der Nutzergruppe - in unterschiedlichen Darstellungsformen eingegeben und auch dargestellt werden. SysMD Notebook und AGILA stellen daher intern alle Modelle in einer Teilmenge von KerML, dem

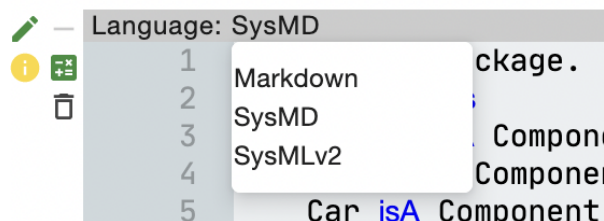
Kern von SysML v2 dar. Auf diese kann über eine in der Standardisierung befindliche REST API zugegriffen werden.

5.1 Unterstützte Sprachen: Markdown, SysMD, SysML v2 Textual

Welche Sprache seitens der Nutzers verwendet wird, ist unerheblich, weil Compiler diese in die interne Darstellung übersetzen. Unterstützt werden aktuell die Sprachen

- Markdown für die Dokumentation
- SysMD
- SysML v2 Textual

Die verwendete Sprache kann im aktuell aktivierten Element in der Menüzeile ausgewählt werden:



Einen ersten Eindruck, wie SysML v2 Textual aussieht kann man unten sehen.

```
Wheel {
  attribute mass: ScalarValue::Real = 70@[SI::kg];
  // model constaint & unit of mass, ...
}

Car :> Vehicle {
  part Wheel [4 .. 8];
  attribute mass = ... // model constraint, unit, ...
}
```

```
Car isA Vehicle.

Car hasA
  wheel: [4 .. 8] Wheel,
  mass: Real(100..1000)kg = sumHasA(mass).

Wheel hasA
  mass: all Real(50 .. 100) kg.
```

Allerdings ist die unterstützte Teilmenge noch klein, der Standard ändert sich und unterstützt nicht alle Features von SysMD. In zukünftigen Versionen wird SysML v2 Textual aber erweitert und an aktuelle Änderungen im Standardisierungsprozess angepasst.